



به نام خدا

موضوع:

بخش تحلیلگر (Report) پروژه UniDB

نویسنده:

عارف زرگر

دی ماه 1404

جدول پیچیدگی زمانی

عملیات	Array Collection	LinkedList Collection	دلیل تفاوت
insertOne (آخر)	$O(1)$	$O(1)$	در آرایه، درج در انتها فقط افزودن عنصر است در لیست پیوندی نیز با داشتن اشاره گر tail، درج انتهایی فقط تغییر چند اشاره گر است.
deleteOne (اول)	$O(n)$	$O(1)$	در آرایه، حذف عنصر اول باعث شیفت تمام عناصر بعدی می شود. اما در لیست پیوندی، فقط head به نود بعدی اشاره می کند.
deleteOne (وسط)	$O(n)$	$O(n)$	در هر دو ساختار ابتدا باید عنصر مورد نظر پیدا شود. در آرایه بعد از حذف، شیفت لازم است و در لیست پیوندی فقط تغییر اشاره گرها انجام می شود، اما جستجو غالب است.
findByID	$O(n)$	$O(n)$	هیچ کدام ساختار ایندکس ندارند، بنابراین جستجوی خطی انجام می شود.
count	$O(1)$	$O(1)$	اگر یک متغیر شمارنده هنگام insert/delete نگهداری شود، شمارش نیازی به پیمایش ندارد.

بنچ مارک (Benchmark)

```
===== BENCHMARK RESULTS (ms) =====  
  
Import 50,000 records | 3732 ms  
Delete 500 from start | 295 ms  
Delete 500 from end | 740 ms  
Search 500 random | 201 ms  
  
=====
```

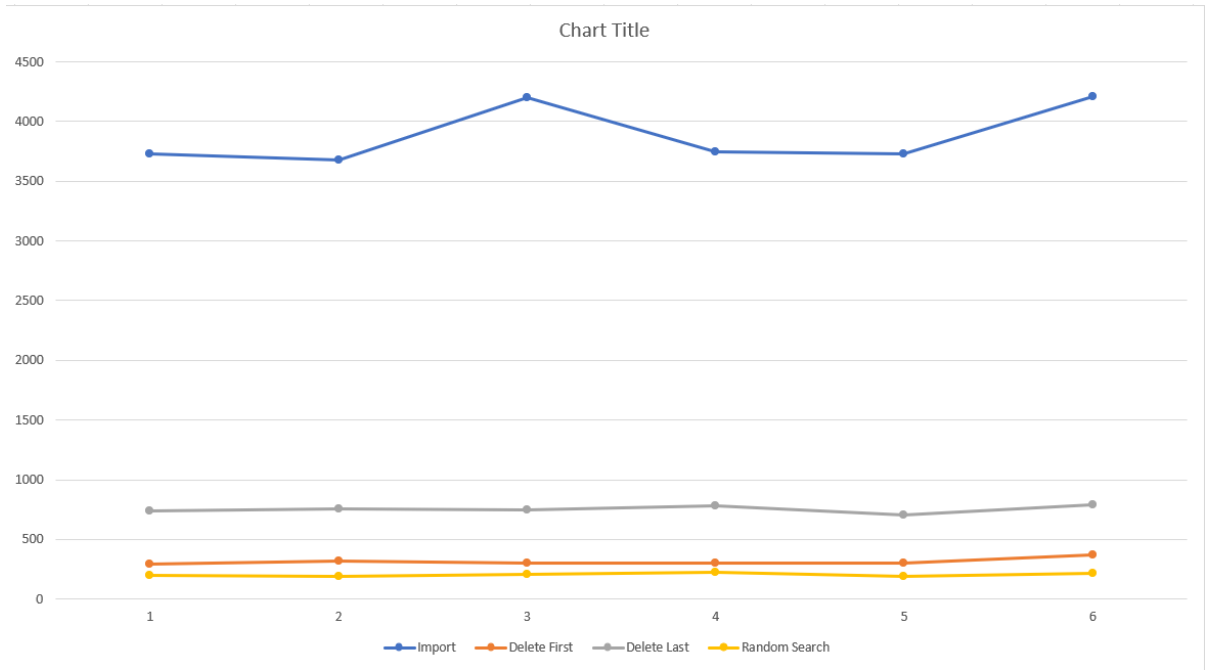
```
===== BENCHMARK RESULTS (ms) =====  
  
Import 50,000 records | 3677 ms  
Delete 500 from start | 316 ms  
Delete 500 from end | 753 ms  
Search 500 random | 193 ms  
  
=====
```

```
===== BENCHMARK RESULTS (ms) =====  
  
Import 50,000 records | 4201 ms  
Delete 500 from start | 301 ms  
Delete 500 from end | 751 ms  
Search 500 random | 205 ms  
  
=====
```

```
===== BENCHMARK RESULTS (ms) =====  
  
Import 50,000 records | 3743 ms  
Delete 500 from start | 305 ms  
Delete 500 from end | 784 ms  
Search 500 random | 222 ms  
  
=====
```

```
===== BENCHMARK RESULTS (ms) =====  
  
Import 50,000 records | 3731 ms  
Delete 500 from start | 305 ms  
Delete 500 from end | 709 ms  
Search 500 random | 192 ms  
  
=====
```

```
===== BENCHMARK RESULTS (ms) =====  
  
Import 50,000 records | 4206 ms  
Delete 500 from start | 371 ms  
Delete 500 from end | 792 ms  
Search 500 random | 214 ms  
  
=====
```



سوالات مفهومی

چرا در لیست پیوندی، `deleteOne(id)` زمان بر است؟

زیرا در لیست پیوندی، دسترسی مستقیم به عناصر وجود ندارد.
برای حذف یک رکورد با `id` مشخص، باید:

1. از `head` شروع کنیم
2. لیست را نود به نود پیمایش کنیم
3. نود موردنظر را پیدا کنیم
4. اشاره‌گرهای قبلی و بعدی را تغییر دهیم

مرحله جستجو غالب است و زمان $O(n)$ دارد، به همین دلیل این عملیات زمان‌بر محسوب می‌شود.

چرا جدا بودن Parser از Storage مفید است؟ (مثلا برای افزودن SQL)

جدا بودن این دو لایه باعث می‌شود:

- Parser فقط مسئول تفسیر دستور باشد
- Storage فقط عملیات داده‌ای را انجام دهد

در نتیجه Parser هیچگاه به Storage دسترسی نخواهد داشت. بنابراین امنیت بالاتر رفته و پروژه قابلیت توسعه و نگهداری دارد.